



Centrifuge PR382 Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

fuzious

Taridoku

LonelySloth

September 14, 2026

Contents

1 About Burra Security 2

2 Disclaimer 2

3 Risk Classification 2

4 Protocol Summary 2

5 Audit Scope 2

6 Executive Summary 3

7 Findings 5

7.1 Informational 5

7.1.1 Hook documentation misstates claim eligibility and checked parties 5

7.1.2 Registrar burn documentation incorrectly states that allowance remains untouched 5

7.1.3 Vault discovery does not validate the requested asset 5

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The in-scope components sit on Centrifuge's spoke side, where a pool's shares are issued, restricted, and settled. `AsyncRequestManager` and `VaultRouter` drive the asynchronous deposit and redemption lifecycle, while `ShareTokenRegistrar` and the transfer hooks own the share token itself and the rules governing who may hold it.

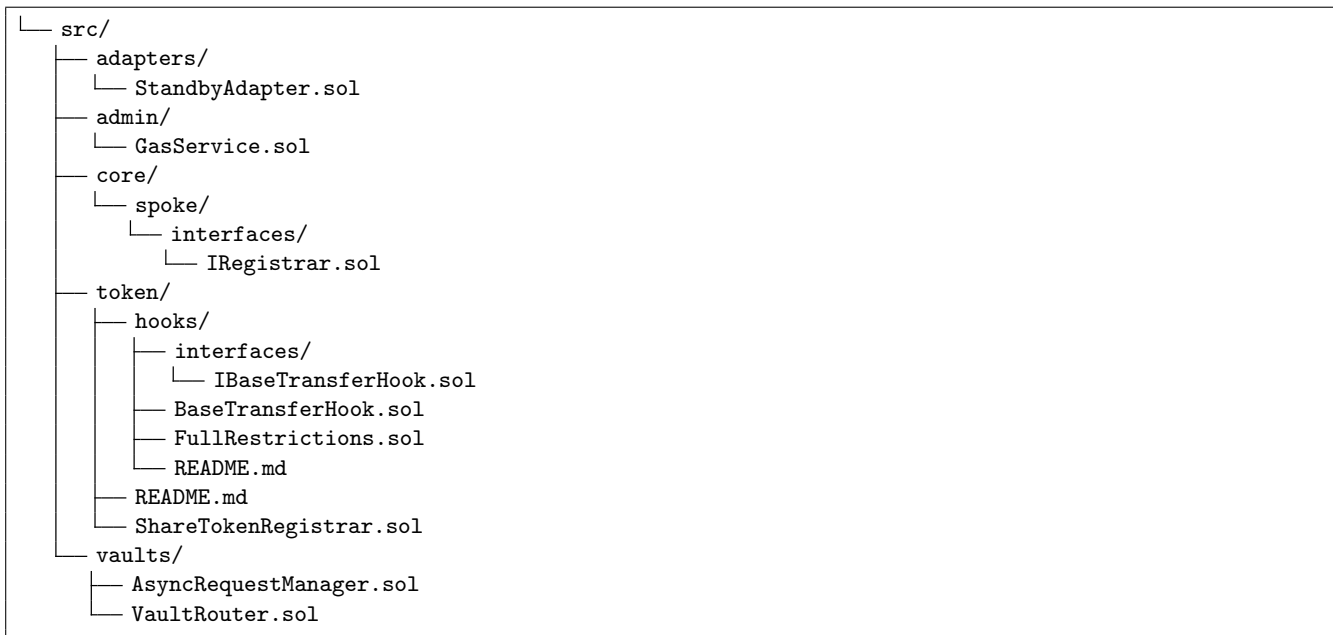
`AsyncRequestManager` receives fulfillment callbacks from the hub, authenticated for a `(poolId, scId, assetId)` tuple, and resolves the vault they act on through the share token's ERC-7575 pointer. `_requestVault` requires that resolved vault to be linked and registered to the same pool, share class, and asset before any investment state is read or written, and `VaultRouter.getVault` applies the same check on the read path. `ShareTokenRegistrar` deploys `ShareToken` deterministically with `CREATE2`, stays a ward on every token it creates, and takes its configuration from the hub through `fromHub`, which swaps transfer hooks, grants wards, and overrides the ERC-7575 pointer against registry storage.

Transfer restrictions live in `BaseTransferHook`, which classifies every mint, burn, and transfer into a lifecycle leg — deposit request, fulfillment, claim, redemption request, cancellation, or cross-chain transfer — and packs memberlist expiry and a freeze bit into the 16 bytes of `hookData` stored alongside each balance. `FullRestrictions` builds on those classifiers to enforce the strictest compliance model, requiring memberlist validity wherever a position enters a holder's hands. `isCrosschainTransfer` matches the pair `ShareTokenRegistrar.canBridge` produces, encoding the destination chain as an address, so outbound bridging is gated on the same membership check as any other issuance.

The share token is an ERC-20 extended with ERC-1404 restriction checks and the ERC-7575 share pointer, settling through ERC-7540 asynchronous vaults. `StandbyAdapter` and `GasService` support the cross-chain messaging layer that carries hub-to-spoke calls, letting an issuer operate share classes across any connected network from a single Hub chain.

5 Audit Scope

The scope of the audit is [PR #382](#), the diff between commit `9b2c1d05d6593b3f273aabc899fae6a9439280bf` and commit `339eb7b18605cbc5b24d04c30a097ff4bdc9092f`, for all the files in the:



6 Executive Summary

Over the course of 2 days, the Burra Security team conducted an audit on the [Centrifuge PR382](#) smart contracts provided by [Centrifuge](#). In this period, a total of 3 issues were found.

Summary

Project Name	Centrifuge PR382
Repository	protocol-internal
Commit	339eb7b18605...
Audit Timeline	Sep 7th - Sep 8th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	3
Gas Optimizations	0
Total Issues	3

Summary of Findings

[I-1] Hook documentation misstates claim eligibility and checked parties	Resolved
[I-2] Registrar burn documentation incorrectly states that allowance remains untouched	Resolved
[I-3] Vault discovery does not validate the requested asset	Resolved

7 Findings

7.1 Informational

7.1.1 Hook documentation misstates claim eligibility and checked parties

Target: [hooks/README.md](#), [IBaseTransferHook.sol](#), [HookClaimLifecycle.t.sol](#)

Severity: Informational

Description: The updated hook documentation states that membership expiry prevents `FreelyTransferable` claims until renewal. However, an expired but unfrozen controller or its authorized operator can direct asset withdrawals and cancelled-deposit refunds to an eligible receiver. Direct `redeem` additionally checks controller eligibility, while deposit claims and cancelled-redemption share refunds check both controller and receiver. A zero self-claim view therefore does not establish that every authorized claim is unavailable. Cancelled-deposit refunds return assets without requiring a share burn. Freeze checks still bind ordinary checked parties, with explicit pool-escrow exemptions.

Recommended Mitigation: Document eligibility per claim entry point, distinguish controller and receiver checks, and clarify self-claim views, cancellation types and escrow exemptions.

Centrifuge: Fixed in [eeb1f90e7062a0f70c1e8879fd3c76507460b430](#)

BurraSec: Fix looks good

7.1.2 Registrar burn documentation incorrectly states that allowance remains untouched

Target: [ShareTokenRegistrar.sol](#)

Severity: Informational

Description: The `ShareTokenRegistrar.burn` comment states that the caller's allowance remains untouched because `sender == from` skips allowance consumption. This applies only to the initial privileged transfer. The subsequent token burn is called by the registrar on behalf of the calling contract, so `ERC20.burn` checks and reduces that contract's finite allowance to the registrar. Unlimited allowance remains unchanged. Integrators relying on the comment could incorrectly expect a finite approval to remain reusable.

Recommended Mitigation: Clarify that the privileged transfer skips allowance consumption, while the subsequent burn consumes finite allowance and preserves unlimited allowance.

Centrifuge: Fixed in [1cb400cf2004923e5dee5880727b81980a1e956b](#)

BurraSec: Fix looks good

7.1.3 Vault discovery does not validate the requested asset

Target: [VaultRouter.sol](#), [SpokeRegistry.sol](#)

Severity: Informational

Description: `getVault(poolId, scId, asset)` checks that the resolved vault is linked and matches the requested pool and share class, but omits asset equality. It can therefore return a vault registered for a different asset within the same pool and share class.

Recommended Mitigation: Add `details.asset == asset` to the nonzero-result validation.

Centrifuge: Fixed in [53871a34ec81fbb011f9eb725922e190e1ac22c0](#)

BurraSec: Fix looks good